

JDBC

jdbc -- java直连数据库

通过驱动，让java可以直接连接对应的数据库进行操作；

Maven

场景：

项目1依赖：abc.jar、spring.jar、log.jar

项目2依赖：spring.jar、log.jar

两个项目需要的依赖不同，且有部分重合；

传统方式：手动下载依赖包，手动配置依赖包；

Maven：项目下不再放jar，而是有一个pom.xml配置文件，描述依赖关系；maven拥有一个本地仓库，如果本地仓库没有，maven就会去远端下载；（类似linux的包管理工具）

配置

1. 下载maven并解压
2. 在根目录下创建一个文件夹\resp
3. 复制\resp的绝对路径
4. 打开conf\settings.xml
5. 找到被注释的,把自己创建的路径写进去

```
<localRepository>D:\neu\apache-maven-3.9.14-bin\apache-maven-3.9.14\resp</localRepository>
```

6. 将/bin写入系统变量中

项目结构

```
src
|--main
|   |--java      # 放源码
|   |--resource  # 放不需要编译的内容
|--test
|   |--java      # 放源码
|   |--resource  # 放资源
|---pom.xml # 重要
```

使用Maven

在pom.xml里写好项目所需要的库

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.neusoft</groupId>
    <artifactId>proj0922</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <dependencies>
        <dependency> <!--在dependency里写明项目所需的依赖-->
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.13</version>
        </dependency>
    </dependencies>
```

```
</project>
```

在pom.xml所在的路径下执行

```
mvn clean install
```

数据库连接

数据库查询 和 数据库增删改 用的不是一个类

数据库查询

```
//1. 加载数据库驱动（postgres驱动）（不同数据库不同的驱动类）
Class.forName("org.postgresql.Driver"); // 这里使用了反射

//2. 定义连接字符串（不同数据库不同写法）
String url = "jdbc:postgresql://localhost:5432/HR_DB";// url，不同的数据库格式有所不同
String user = "postgres"; // 用户名
String pass = "passwd"; // 密码

//3.创建数据库的连接
Connection conn = DriverManager.getConnection(url, user, pass);
System.out.println(conn);

//4.声明要执行的SQL语句
String sql = "SELECT * FROM dept";

//5.创建一个Statement用于加载SQL
PreparedStatement pst = conn.prepareStatement(sql);

//6.声明结果集对象获取SQL语句的执行结果
ResultSet rs = pst.executeQuery();

//7.迭代遍历数据
while (rs.next()) {
    //取数据操作
    int deptno = rs.getInt("deptno"); //结果集名字是“deptno”列
    String dname = rs.getString("dname"); //结果集的第2列
    int locid = rs.getInt("locid");
```

```

        //Date hiredate = rs.getDate("hiredate");
        System.out.println(deptno + "\t" + dname + "\t" + locid);
    }
    //8.依次关闭ResultSet, PreparedStatement, Connection
    rs.close();
    pst.close();
    conn.close();

```

数据库增删改

```

//1. 加载数据库驱动（postgres驱动）（不同数据库不同的驱动类）
Class.forName("org.postgresql.Driver");

//2. 定义连接字符串（不同数据库不同写法）
String url = "jdbc:postgresql://localhost:5432/HR_DB";
String user = "postgres";
String pass = "passwd";

//3.创建数据库的连接
Connection conn = DriverManager.getConnection(url, user, pass);

//4.声明要执行的SQL语句
String sql = "INSERT INTO dept VALUES(?, ?, ?)";

//5.声明Statement加载SQL
PreparedStatement pst = conn.prepareStatement(sql);
//6.为? 依次绑定数据
int deptno = 50;
String dname = "INFO";
int locid = 100;

pst.setInt(1, deptno); //将deptno的值绑定给SQL中的第一个?
pst.setString(2, dname);
pst.setInt(3, locid);

//pst.setDate(4, new java.sql.Date(System.currentTimeMillis()));
//7.执行DML操作

```

